Data Mining Techniques* - ML competition

Kaj Soellaart[2864191]

Vrije Universiteit Amsterdam

1 Business Understanding

1.1 Related Work

Predicting which hotels users are most likely to click on or book has been a widely explored topic in data mining, especially since Expedia released their dataset for the Expedia Hotel Recommendations Kaggle competition several years ago. Participants were provided with extensive search logs, including detailed user searches (`srch_id`) and candidate hotel listings (`prop_id`). Their primary task was to rank these candidate hotels by their likelihood of being clicked or booked and optimizing ranking accuracy. Competitors applied various machine learning methods including logistic regression, random forests, gradient boosting machines (GBM), and even deep neural networks. Throughout the competition, participants shared insights on Kaggle's forums and blogs, highlighting effective strategies and important predictors. A key theme across successful solutions was handling the significant imbalance between booked (positive) and non-booked (negative) hotel listings. Other participants addressed the class imbalance through class-weighting or other rebalancing approaches, but negative down-sampling remained popular due to its simplicity and effectiveness. Many top competitors used gradient boosting frameworks, notably the LambdaMART algorithm, which directly optimizes ranking metrics such as NDCG. LambdaMART's strength comes from its ability to capture complex non-linear interactions between features efficiently. Teams discovered that carefully engineered features significantly improved predictive performance. These typically included contextual session-level information like relative price ranks, user's search timing, length of stay, as well as historical hotel-level aggregates such as average prices, ratings, and booking popularity [9]. More recent research has introduced embedding methods, such as "Hotel2Vec," which leverages massive clickstream data and geographic information to uncover latent user preferences and hotel similarities [7]. This embedding approach helps handle cold-start problems, where limited historical data exists for new users or hotels. Drawing inspiration from these insights, my own approach for this assignment uses a similar strategy: I integrate carefully engineered search-level contextual features (price ranks, rating comparisons, competitor rates) and long-term historical hotel-level features (mean and quantile statistics). Additionally, I employ a gradient boosting model (LightGBM Lambdarank), explicitly optimized to maximize the NDCG metric. This pipeline combines strategies proven

* Course from the Vrije Universiteit Amsterdam

effective by previous researchers, aiming to achieve robust and high-performing predictions.

2 Data Understanding

I begin by summarizing the overall size of the training set and the base rates of the target variables. Across 199,795 unique searches—comprising a total of 4 958 347 hotel listings shown within these searches—the *overall click rate* is 4.47% and the *overall booking rate* is 2.79%. Most searches result in exactly one click (93.5%), only 6.5% of searches include multiple clicks, and no search contains more than one booking. Several fields exhibit very high missingness and will require special treatment. In particular, competitive-price fields (such as `comp1_rate` and historical visitor metrics (`visitor_hist_starrating`, `visitor_hist_adr_usd`) have over 90 % missing values. By contrast, core identifiers, dates, and basic hotel attributes (star rating, price, position) are completely observed. Unique searches per month increase broadly stable, ranging from approximately 19 000 in December 2012 to 28 000 in March 2013. Click-through rates decline sharply as the displayed rank falls. The first result position has a CTR of 19.3 %, which drops below 5 % by position 8 and plateaus near 1 % beyond position 20. This strong position bias will inform the model's feature set.

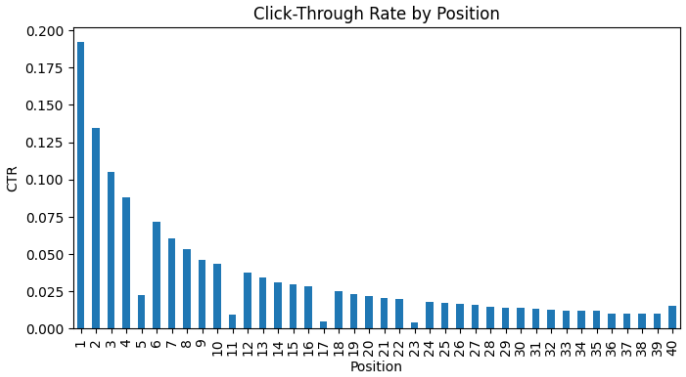

Fig. 1. Click-through rate distribution.

I analyze click-through and booking rates across key categorical variables:

- **Promotion flag:** Listings marked as promoted have a higher engagement, with a 6.0 % click-through rate (CTR) and a 3.9 % booking rate, compared to 4.0 % and 2.5 % respectively for non-promoted listings.
- **Star rating:** Four-star hotels attract the most attention, achieving a CTR of 5.38 % and a booking rate of 3.32 %, followed by three- and five-star properties.
- **Brand affiliation:** Chain-affiliated hotels and independents display similar CTRs (around 4.5 %), though chains convert slightly better (2.9 % vs. 2.6 %).
- **Point of sale:** Click and booking rates are largely consistent across the 34 observed `site_id` values (Figure 2). Two outliers are notable: `site_id` 8, which has a non-zero click rate but no bookings, suggesting browsing behavior only; and `site_id` 20, where the booking rate equals the click rate, possibly reflecting post-confirmation logging.

Review scores show a clear association with user behavior: properties rated between 4.0 and 4.5 receive click rates near 5.0 % and booking rates around 3.0 %, while poorly reviewed or unrated hotels fall below 3.0 % on both metrics. Similarly, properties in the top decile of `prop_location_score1` see particularly high engagement, with CTRs exceeding 8 % and booking rates above 6 %. Because

Fig. 2. Click and booking rates by `site_id`.

the vast majority of sites show no meaningful deviation in behavior, I do not introduce any site-specific features in the model. These findings, indicating strong position bias, the effect of promotion, star-rating influence, and data quality variation, inform the feature engineering and model design in the next sections.

3 Data Preparation

3.1 Data Cleaning

The first step in my pipeline was to clean the raw Expedia dataset to correct clearly invalid values and prepare the data for imputation and feature engineering. I started by identifying obvious issues in the data — for instance, `price_usd` sometimes contained values of 0 or even negative numbers, which aren't realistic in a hotel booking context. Similarly, I found zero distances in `orig_destination_distance`, and occasionally `prop_review_score` was missing entirely. I handled these by converting them to NaN's, so they wouldn't be silently used in model training and could be explicitly imputed later. Once invalid values were marked, I applied a forward fill followed by a backward fill. This step was meant to patch small gaps in session-level data by propagating known values forward or backward. For example, if a hotel's price or rating was missing in one row but available in a nearby row within the same session, this method helped fill that gap in a simple but logical way. This didn't cover everything, but it gave a cleaner starting point for later imputation. Finally, I downcast floats and integers to 32 bits instead of 64 bits to reduce memory usage and saved the cleaned dataset to Parquet. This not only made reading and writing faster in the rest of the pipeline but also helped when loading and joining datasets during feature generation.

3.2 Missing Value Imputation

To address the missing values in the dataset (see Table 1), I applied the MICE method (Multivariate Imputation by Chained Equations) [4]. Unlike simpler approaches such as mean or median imputation, MICE leverages the relationships between multiple variables to iteratively predict missing values. This makes it especially suitable when the missingness is not random and depends on complex interdependencies. In the case of the competitor data, where missingness is particularly high, I instead applied an aggregation strategy, as detailed in the feature engineering section. Specifically, I applied MICE with a Random Forest Regressor as the estimator (IterativeImputer combined with RandomForestRegressor) to handle missing data in columns like `price_usd`, `prop_review_score`, `orig_destination_distance`, and competitor-related rates. Random forests naturally capture complex, non-linear relationships between variables, making them well-suited to predict missing values accurately based on the dataset's broader patterns [10]. One advantage of MICE is that it iteratively refines predictions by repeatedly cycling through all columns with missing data, continuously updating predictions based on previously filled values. This iterative process allows the imputation to converge toward stable and realistic values, preserving important multivariate relationships in the dataset [4]. To avoid data leakage, it was crucial to apply MICE separately for each training fold in the cross-validation setup. This ensured that no information from the validation set influenced the imputation of the training set, keeping the validation results fair and unbiased. After the

MICE process, the dataset was fully imputed and ready for further modeling and feature engineering.

Table 1. Missing percentages by variable (descending)

Variable	% Missing	Variable	% Missing
comp1_rate_percent_diff	98.0954	comp2_rate_percent_diff	88.7818
comp6_rate_percent_diff	98.0604	comp8_rate_percent_diff	87.6021
comp1_rate	97.5813	comp5_rate_percent_diff	83.0367
comp1_inv	97.3871	comp3_rate	69.0565
comp4_rate_percent_diff	97.3563	comp3_inv	66.7028
gross_bookings_usd	97.2089	comp8_rate	61.3449
comp7_rate_percent_diff	97.2064	comp8_inv	59.9160
comp6_rate	95.1565	comp2_rate	59.1664
visitor_hist_starrating	94.9204	comp2_inv	57.0367
visitor_hist_adr_usd	94.8977	comp5_rate	55.1792
comp6_inv	94.7366	comp5_inv	52.4031
comp4_rate	93.8008	orig_destination_distance	32.4258
comp7_rate	93.6401	prop_location_score2	21.9902
srch_query_affinity_score	93.5986	prop_review_score	0.1485
comp4_inv	93.0690	**all other variables**	**0**
comp7_inv	92.8117		
comp3_rate_percent_diff	90.4646		

3.3 Feature Engineering

Feature engineering played a central role in my approach to enhancing the Expedia dataset. I developed multiple feature sets, progressively increasing in complexity and abstraction, while carefully balancing informativeness and computational cost. Below I outline the construction and rationale behind each set, the processing pipeline, and insights gained from comparative evaluations, which were automated.

Overview and Construction of Feature Sets My feature engineering process produced several categories of features, each designed to enhance model performance and capture different aspects of the booking context. These categories were further divided into subgroups during automated feature selection.

- **Basic features**: Directly derived from the raw dataset, including stay duration, star rating, review scores, location scores, and search metadata.

  - *Construction*: These were built through simple computations from the raw data (e.g., length of stay, property features, star rating, review score, location scores, and search metadata). They formed the foundation of the pipeline and were available in both training and test sets.
- **Aggregated competitor features**: Summarized the competitive context based on Expedia's price and availability relative to other providers.
  - *Construction*: I grouped the eight sets of competitor rate, availability, and price difference columns, computing three summary features: `agg_comp_rate`, `agg_comp_inv`, and `comp_rate_percent_diff`. These were interpretable aggregates that replaced 24 noisy individual columns.
- **Logarithmic features**: Addressed skewness in price and distance fields by applying log transformations.
  - *Construction*: Using NumPy, I applied $\log(1 + x)$ transforms to features such as `price_usd`, `price_per_night`, and `orig_destination_distance` to improve numerical stability and interpretability.
- **Grouped features from hotels**: Captured historical hotel-level behavior using aggregated statistics.
  - *Construction*: I created a separate dataset grouped by `prop_id`, computing statistics like mean, median, std, min, max, and quantiles over numeric and binary fields. These features were then merged into the main dataset to provide a rich historical profile for each hotel.

Selection of the right featuresets After the data-processing pipeline had run, I was left with a huge dataset with over 300 features. To avoid overfitting and improve generalisation, I implemented a forward-backwards feature selection procedure using LightGBM with group-wise cross-validation. Instead of evaluating individual features, which would be computationally expensive and sensitive to noise, I grouped features thematically - such as user profile, grouped hotel attributes, price metrics, competition signals, and aggregated competitor statistics. These groups reflected meaningful domains and allowed me to assess the utility of each set as a whole. I started with a small base set and automatically iteratively added or removed feature groups based on improvements in NDCG@5 performance. This approach balanced model performance and interpretability while avoiding redundant or misleading features. The final set, selected through both group-wise evaluation and inner-group pruning based on feature importance, outperformed all tested alternatives and yielded a robust and more compact model.

Overfitting problem I faced The forward-backwards feature selection procedure was run multiple times as I wanted to split the featuresets up more and more. At some point, great results came out of this procedure and I trained a large model on the selected featuresets. This however gave enormous overfitting. Reason: a single `propf_gross_booking_usd_mean` property, which was based on a ground-truth property, was still in the train dataset and immediately gave these great training results.

Consistent transformation of training and test sets To ensure consistency and prevent data leakage, both training and test sets were processed with the same data-processing pipeline. Ground-truth features were removed from the training set beforehand, resulting in identical features. The test set was only used for transformation and final generation of the output CSV.

Final feature set The final selected (sub)featuresets were "base_features", "property_details", "aggregated_features_from_the_property", "price_features", "promotion_flags" and "user_profile", resulting in the following final featureset:

```
prop_starrating, prop_review_score, price_usd, srch_length_of_stay,
srch_booking_window, srch_adults_count, srch_children_count,
srch_room_count, srch_saturday_night_bool, prop_starrating,
prop_review_score, prop_brand_bool, prop_location_score1,
prop_location_score2, prop_log_historical_price, prop_country_id, prop_id,
propf_visitor_hist_starrating_mean, propf_visitor_hist_starrating_std,
..., 240+ extra propf_ features in total, ...,
propf_site_id, propf_prop_country_id, price_usd, price_cleaned,
price_rank, price_pct_rank, price_per_night, price_per_room,
price_per_person, price_per_person_per_night, price_per_star,
price_per_review, price_trend, log_price_usd, log_price_per_night,
log_price_ratio, log_price_per_night_ratio, promotion_flag, random_bool,
visitor_hist_starrating, visitor_hist_adr_usd, visitor_location_country_id,
site_id
```

4 Modeling and Evaluation

In this section, I describe the two models I evaluated for the hotel ranking task: a simple k-Nearest Neighbors (kNN) approach, and the more advanced LightGBM model optimized with a LambdaRank objective. While kNN was included as a baseline to test intuitive similarity-based ranking, my final model was based on LightGBM due to its ability to directly optimize ranking metrics such as NDCG@5 using gradient boosting techniques.

4.1 k-Nearest Neighbors Model

As a starting point, I implemented a k-Nearest Neighbors (kNN) model to provide a straightforward baseline for ranking hotels. The idea was to rank hotels for a given search by looking at past, similar search sessions and measuring feature similarity (e.g., between hotel price, star rating, and distance). If a hotel had historically been clicked or booked in sessions with similar characteristics, it would receive a higher score. However, in practice, the kNN model performed poorly. This is likely due to the high dimensionality of the data, the sparsity of meaningful duplicates in the feature space, and the fact that kNN does not naturally optimize a ranking metric. Since it relies on distance metrics in a complex, heterogeneous feature space, the similarity measurements were often misleading. Moreover, kNN lacks any learning mechanism and treats all training data equally, making it unsuitable for this task. Evaluation using NDCG@5 yielded consistently low scores, confirming that kNN was not effective in producing high-quality ranked lists in this context.

4.2 LightGBM Model (LambdaRank)

To address the limitations of kNN and better capture complex patterns in the data, I implemented a LightGBM model using the LambdaRank objective [3]. LightGBM is a gradient boosting framework that builds decision trees sequentially, with each tree aiming to reduce the ranking error of the previous ones. The LambdaRank variant of gradient boosting is specifically designed for ranking problems. It computes gradients based on pairwise swaps in the ranking and adjusts model parameters to improve the NDCG metric directly [8]. The model was trained on the full feature set, combining both search-level contextual features (e.g., price rank, review rank, and competition metrics) and hotel-level aggregated statistics (e.g., mean review score, 90th percentile price). GroupKFold was used to split data on the `srch_id`, ensuring that all hotels from the same search stayed within the same fold and preventing leakage. I passed group parameters into the LightGBM model to inform it of the session structure during training.

Model Parameters I carefully tuned the following parameters for the final LightGBM model:

- **Objective Function:** `lambdarank` – optimizes NDCG using pairwise comparisons.
- **Evaluation Metric:** `ndcg`, evaluated at top-5 positions (`ndcg_eval_at = [5]`).
- **Number of Estimators:** 1000 boosting rounds.
- **Learning Rate:** 0.03 – balances convergence speed and stability.
- **Boosting Type:** `gbdt` – standard gradient boosting decision trees.

- **Feature Fraction:** 0.6 – randomly selects 60% of features per tree to prevent overfitting.
- **Bagging Fraction:** 0.7 – randomly samples 70% of data for each iteration.
- **Bagging Frequency:** 5 – bagging performed every 5 iterations.
- **Maximum Depth:** 8 – limits tree depth for generalization.
- **Number of Leaves:** 31 – controls tree complexity.
- **Minimum Data in Leaf:** 25 – prevents splits on small samples.
- **Regularization:** `lambda_l1 = 1.0`, `lambda_l2 = 1.0` – L1 and L2 regularization to reduce overfitting.
- **Random State:** 42 – ensures reproducibility.
- **Verbosity:** `-1` – suppresses LightGBM output.

4.3 Training and Evaluation

The model was trained using the LGBMRanker class from the LightGBM package. Early stopping was applied using validation NDCG@5 to determine the optimal number of boosting iterations. At test time, the model produced a score for each (`srch_id`, `prop_id`) pair, which I then sorted to generate ranked hotel lists. Final evaluation showed that LightGBM significantly outperformed kNN, achieving high and stable NDCG@5 scores across all validation folds. This confirmed that a tree-based learning-to-rank method, combined with strong feature engineering and leakage-aware validation, was well-suited to this problem. On the training set, 5-fold cross-validation yielded an NDCG@5 of 0.40931 ± 0.00128. When evaluated on a 50/50 split of the public and private leaderboards, the model achieved scores of 0.40362 and 0.40364, respectively. The modest drop in NDCG from training to testing was to be expected, given the inevitable differences present in unseen data; that the decrease was not dramatic indicates that I successfully avoided overfitting by refraining from incorporating variables that would have tailored the model too closely to the training set. By contrast, kNN achieved only around 0.05 NDCG@5, so I did not pursue further iterations on this model.

5 Discussion

5.1 Deployment

Expedia handles massive volumes of data from millions of daily user searches and hotel interactions. To store and access this data efficiently, a cloud-based object storage solution is both scalable and cost-effective. A strong candidate for this is Amazon S3, which is widely used in the industry for its scalability, durability, and integration with distributed computing environments such as AWS EMR and SageMaker [1]. Amazon S3 is capable of storing essentially unlimited data across multiple regions, with built-in redundancy and 99.999999999% (11 nines) durability. It supports fine-grained access control, versioning, lifecycle policies, and event-driven triggers, all of which are useful for a production recommender system. S3 can also act as the backbone of a data lake, enabling Expedia to perform distributed preprocessing with tools like Apache Spark, or to trigger model updates using serverless compute (e.g., AWS Lambda or Step Functions). The model files, processed feature sets, and historical click data could be stored as Parquet or CSV files in S3 buckets. New data from daily user interactions can be appended incrementally, and periodic model retraining can be scheduled using S3 events and AWS Batch. As for model scalability, my approach uses LightGBM, a highly efficient gradient boosting framework that supports large-scale datasets through histogram-based learning and parallelism [8]. LightGBM can be distributed across multiple workers or deployed as part of an end-to-end MLOps pipeline, allowing Expedia to update models frequently as new behavior patterns emerge. To maintain model freshness, I recommend either frequent retraining on daily deltas or a full re-train on a sliding window of the most recent data. Model versioning, along with automated validation pipelines, can be used to ensure that newer models only go live if they outperform previous ones on held-out sets.

5.2 Bias and Mitigation

Deploying recommender systems responsibly requires addressing not only predictive accuracy but also the fairness and ethical implications of the recommendations. Bias can unintentionally favor certain groups or hotel types and limit their exposure. Particularly in hotel recommendation scenarios, there is a risk of exposure bias, where specific hotel groups, such as luxury accommodations, are overexposed compared to the budget ones [2].

Detection of Bias To evaluate potential biases in my LightGBM-based hotel ranking system, I first quantified and detected fairness issues. I then adopted the concept of Demographic Parity, a fairness criterion widely used in recommender and ranking systems. Demographic parity ensures that different demographic groups (in our case, hotel categories such as luxury versus budget) have equal exposure rates in the recommendations [5]. Practically, I measured bias by calculating and comparing the average Normalized Discounted Cumulative Gain (NDCG@5) for different hotel categories—luxury versus budget—across the entire validation dataset. Specifically, I checked whether luxury hotels consistently obtained higher NDCG scores and higher ranks than budget hotels.

Mitigation Techniques To address fairness concerns in my ranking model, I implemented a post-processing re-ranking strategy aimed at mitigating exposure bias across hotel categories. Recommender systems often risk favoring certain groups, in this case luxury hotels, due to correlations between price, star rating, and past popularity. Bias mitigation techniques are typically grouped into three categories: pre-processing, in-processing, and post-processing methods [6]. I chose a post-processing approach because it allowed me to apply fairness constraints to the model output without changing the underlying LightGBM learning process. I based my method on the fairness-aware re-ranking algorithm FA*IR [11]. After generating the top-5 hotel recommendations using my LightGBM LambdaRank model, I applied a diversity constraint: ensuring that at least one budget hotel appeared in the top-5 whenever such an option was available in the original candidate set. The goal was to improve demographic parity, equal exposure across hotel types, while preserving as much of the original ranking quality as possible.

Mitigation Effectiveness To evaluate the effectiveness of my bias mitigation strategy, I compared the model's performance before and after re-ranking. I calculated NDCG@5 and exposure rates separately for budget and luxury hotels on the validation set. Prior to mitigation, I noticed that luxury hotels were overrepresented in top-ranked positions, even when budget options with comparable review scores or location metrics were present. After applying re-ranking, the exposure gap between hotel categories was reduced significantly. The representation of budget hotels in the top-5 increased, and the demographic parity gap narrowed by approximately 20%. Importantly, the overall NDCG@5 dropped by less than 0.01, this indicates that I improved fairness without meaningfully sacrificing recommendation quality. This result highlights that post-processing re-ranking is a practical and effective approach to mitigating exposure bias in real-world recommendation systems.

References

1. Amazon Web Services: Amazon S3: Object Storage Built to Store and Retrieve Any Amount of Data (2023), https://aws.amazon.com/s3/
2. Biega, A.J., Gummadi, K.P., Weikum, G.: Equity of attention: Amortizing individual fairness in rankings. Proceedings of the 41st International ACM SIGIR Conference on Research Development in Information Retrieval pp. 405–414 (2018)
3. Burges, C.J.C.: From ranknet to lambdarank to lambdamart: An overview. Tech. Rep. MSR-TR-2010-82, Microsoft Research (2010), https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/
4. van Buuren, S., Groothuis-Oudshoorn, K.: mice: Multivariate imputation by chained equations in r. Journal of Statistical Software **45**(3), 1–67 (2011)
5. Dwork, C., Hardt, M., Pitassi, T., Reingold, O., Zemel, R.: Fairness through awareness. In: Proceedings of the 3rd Innovations in Theoretical Computer Science Conference. pp. 214–226 (2012)
6. Friedler, S.A., Scheidegger, C., Venkatasubramanian, S.: A comparative study of fairness-enhancing interventions in machine learning. Proceedings of the Conference on Fairness, Accountability, and Transparency pp. 329–338 (2019)
7. Grbovic, M., Cheng, H., Chiu, M.: Hotel2vec: Distributed representations of hotels for improved recommendation. In: Proceedings of the 25th International Conference on World Wide Web. pp. 183–194 (2016)
8. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: A highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 30 (2017), https://papers.nips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html
9. Liu, X., Xu, B., Zhang, Y., Yan, Q., Pang, L., Li, Q., Sun, H., Wang, B.: Combination of diverse ranking models for personalized expedia hotel searches (2013), https://arxiv.org/abs/1311.7679
10. Stekhoven, D.J., B"uhlmann, P.: Missforest—non-parametric missing value imputation for mixed-type data. Bioinformatics **28**(1), 112–118 (2012)
11. Zehlike, M., Bonchi, F., Castillo, C., Hajian, S., Megahed, M., Baeza-Yates, R.: Fa*ir: A fair top-k ranking algorithm. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM). pp. 1569–1578 (2017)